

MERN Stack Real-Time Chatroom & Socket.io Guide

Complete Codebase Implementation, Dynamic Link Generation, and Detailed Architectural Explanations

1. System Architecture Overview

This document provides a comprehensive, production-ready blueprint for implementing real-time, room-isolated chatrooms in a MERN (MongoDB, Express, React, Node.js) application using **Socket.io** and dynamic routing with **React Router**.

Key System Capabilities:

- **Dynamic Room Generation:** Users can click a 'Create Chatroom' button on the GameList page to open a modal.
- **Unique Shareable Links:** The app generates a unique URL (e.g. `http://localhost:5173/chatroom/room-1722591234-a1b2c3`).
- **Real-Time Isolated Messaging:** Socket.io rooms enable instant messaging strictly between users connected to the exact same room ID.
- **Protected Navigation:** Integrates with React Router and PrivateRoute so unauthenticated users are directed to login first.

2. Required Package Dependencies

To enable Socket.io communication, install the corresponding packages in your backend and frontend projects:

Backend Terminal Command:

```
cd backend
npm install socket.io
```

Frontend Terminal Command:

```
cd ChronicleArcade
npm install socket.io-client
```

3. Complete Code Implementation & Explanations

Component A: CreateChatroomModal.jsx

This component renders a modal popup where users type a chatroom title. It generates a unique room ID using timestamps and random string hashes, formats a shareable link, and provides options to copy the link or immediately navigate into the chatroom.

```
// File: ChronicleArcade/src/components/CreateChatroomModal.jsx
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';

export function CreateChatroomModal({ isOpen, onClose, gameTitle }) {
  const [roomName, setRoomName] = useState('');
  const [generatedLink, setGeneratedLink] = useState('');
  const navigate = useNavigate();

  if (!isOpen) return null;

  const handleCreateRoom = (e) => {
    e.preventDefault();
    if (!roomName.trim()) return;

    const roomId = `room-${Date.now()}-${Math.random().toString(36).substr(2, 6)}`;
    const fullLink = `${window.location.origin}/chatroom/${roomId}`;
    setGeneratedLink(fullLink);
  };

  const handleCopyLink = () => {
    navigator.clipboard.writeText(generatedLink);
    alert('Chatroom link copied to clipboard!');
  };

  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/70 backdrop-blur-sm p-4">
      <div className="w-full max-w-md bg-[#12121f] border border-slate-700 rounded-2xl p-6 shadow-2xl relative text-white">
        <button onClick={onClose} className="absolute top-4 right-4 text-slate-400 hover:text-white text-lg font-bold">
          X
        </button>
        <h3 className="text-xl font-bold text-cyan-400 mb-2">Create Chatroom</h3>
        <p className="text-xs text-slate-400 mb-4">Game: {gameTitle} || 'Arcade Game'</p>

        <!-- Form for creating chatroom -->
        <!-- !generatedLink ? ( -->
        <form onSubmit={handleCreateRoom} className="flex flex-col gap-4">
          <div>
            <label className="text-xs text-slate-300 mb-1 block">Chatroom Name</label>
            <input
              type="text"
              placeholder="Enter room name (e.g. Squad Tactics)"
              value={roomName}
              onChange={(e) => setRoomName(e.target.value)}
              className="w-full p-3 bg-[#0a0a14] border border-slate-700 rounded-lg text-sm text-white outline-none focus:ring-1 focus:ring-cyan-400"
              required
            />
          </div>
          <button type="submit" className="w-full py-2.5 bg-cyan-400 text-black font-bold text-sm rounded-lg hover:bg-cyan-500">
            Generate Chatroom Link
          </button>
        </form>
        <!-- generatedLink ? ( -->
      </div>
    </div>
  );
}
```

```
        <button onClick={handleCopyLink} className="flex-1 py-2 bg-slate-800 border border-slate-600 rounded-lg t
            ■ Copy Link
        </button>
        <button onClick={() => { onClose(); navigate(new URL(generatedLink).pathname); }} className="flex-1 py-2
            ■ Join Room
        </button>
    </div>
</div>
    )}
</div>
</div>
    );
}
```

Component B: GameList.jsx Integration

Integrates the 'Create Chatroom' button inside GameList, maintaining state for modal visibility.

```
// File: ChronicleArcade/src/components/GameList.jsx (Excerpt)
import { useState } from 'react';
import { CreateChatroomModal } from '../CreateChatroomModal';

export function GameList() {
  const [isModalOpen, setIsModalOpen] = useState(false);

  return (
    <div className="p-6">
      <div className="flex justify-between items-center mb-6">
        <h1 className="text-2xl font-bold text-white">Featured Games</h1>
        <button
          onClick={() => setIsModalOpen(true)}
          className="px-4 py-2 bg-cyan-400 text-black font-bold rounded-lg text-sm hover:bg-cyan-300 transition-all cur
        >
          ■ Create Chatroom
        </button>
      </div>

      <CreateChatroomModal
        isOpen={isModalOpen}
        onClose={() => setIsModalOpen(false)}
        gameTitle="Chronicle Game"
      />
    </div>
  );
}
```

Component C: ChatroomPage.jsx (Real-Time Socket Client)

Reads the dynamic roomId from the URL params using React Router's useParams() hook. Connects to Socket.io, emits a join-room event, listens for incoming real-time messages, and emits messages sent by the user.

```
// File: ChronicleArcade/src/screen/ChatroomPage.jsx
import { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import { io } from 'socket.io-client';

const socket = io('http://localhost:5174', {
  withCredentials: true,
  autoConnect: false,
});

export function ChatroomPage() {
  const { roomId } = useParams();
  const [messages, setMessages] = useState([]);
  const [inputMessage, setInputMessage] = useState('');
  const currentUser = localStorage.getItem('username') || 'Gamer';

  useEffect(() => {
    socket.connect();
    socket.emit('join-room', roomId);

    socket.on('receive-message', (data) => {
      setMessages((prev) => [...prev, data]);
    });

    return () => {
      socket.off('receive-message');
      socket.disconnect();
    };
  });
}
```

```

}, [roomId]);

const handleSendMessage = (e) => {
  e.preventDefault();
  if (!inputMessage.trim()) return;

  socket.emit('send-message', {
    roomId,
    message: inputMessage,
    sender: currentUser,
  });

  setInputMessage('');
};

return (
  <div className="max-w-3xl mx-auto my-8 p-6 bg-[#12121f] rounded-2xl border border-slate-700 text-white">
    <h2 className="text-xl font-bold text-cyan-400 mb-1">Chatroom: {roomId}</h2>
    <p className="text-xs text-slate-400 mb-6">Live real-time room messaging</p>

    <div className="h-80 overflow-y-auto p-4 bg-[#0a0a14] rounded-lg border border-slate-800 flex flex-col gap-3 mb-4">
      {messages.length === 0 ? (
        <p className="text-xs text-slate-500">No messages in this room yet. Be the first to chat!</p>
      ) : (
        messages.map((msg, idx) => (
          <div key={idx} className="p-2 bg-slate-800/60 rounded-md text-sm">
            <span className="text-cyan-400 font-bold text-xs mr-2">{msg.sender}</span>
            <span className="text-slate-200">{msg.message}</span>
          </div>
        ))
      )}
    </div>

    <form onSubmit={handleSendMessage} className="flex gap-2">
      <input
        type="text"
        placeholder="Type your message..."
        value={inputMessage}
        onChange={(e) => setInputMessage(e.target.value)}
        className="flex-1 p-3 bg-[#0a0a14] border border-slate-700 rounded-lg text-sm text-white outline-none focus:outline-1 focus:outline-cyan-400">
      </input>
      <button type="submit" className="px-5 py-3 bg-cyan-400 text-black font-bold text-sm rounded-lg hover:bg-cyan-300">
        Send
      </button>
    </form>
  </div>
);
}

```

Backend Implementation: app.js (HTTP + Socket.io Server)

Replaces standard Express app.listen with Node's http.createServer(app). Initializes Socket.io with CORS credentials, handles join-room events to isolate room traffic, and broadcasts messages using io.to(roomId).emit(...).

```
// File: backend/app.js
import 'dotenv/config';
import cors from 'cors';
import express from 'express';
import cookieParser from 'cookie-parser';
import { createServer } from 'http';
import { Server } from 'socket.io';
import { connectDB } from './db.js';

connectDB();
const app = express();
const httpServer = createServer(app);

const io = new Server(httpServer, {
  cors: {
    origin: ['http://localhost:5173', 'http://localhost:5174', 'http://localhost:3000'],
    credentials: true,
  },
});

app.use(cors({
  origin: ['http://localhost:5173', 'http://localhost:5174', 'http://localhost:3000'],
  credentials: true,
}));
app.use(cookieParser());
app.use(express.json());

io.on('connection', (socket) => {
  socket.on('join-room', (roomId) => {
    socket.join(roomId);
  });

  socket.on('send-message', ({ roomId, message, sender }) => {
    io.to(roomId).emit('receive-message', {
      sender,
      message,
      createdAt: new Date().toISOString(),
    });
  });
});

const PORT = process.env.PORT || 5000;
httpServer.listen(PORT, () => {
  console.log(`Server and Socket.io running on http://localhost:${PORT}`);
});
```

App.jsx Route Declarations

Defines dynamic route matching /chatroom/:roomId using React Router:

```
// File: ChronicleArcade/src/App.jsx (Excerpt)
import { Route, Routes } from 'react-router-dom';
import { ChatroomPage } from './screen/ChatroomPage';
import { PrivateRoute } from './components/PrivateRoute';

<Routes>
  {/* Existing routes */}
  <Route
    path="/chatroom/:roomId"
    element={<PrivateRoute><ChatroomPage /></PrivateRoute>}
  />
```

```
    />  
  </Routes>
```

4. Architectural Principles & Best Practices

- 1. Room Isolation:** Calling `socket.join(roomId)` groups sockets into logical channels on the server. Using `io.to(roomId).emit()` guarantees that chat traffic from room-A is never received by users in room-B.
- 2. URL Parameters & State Sync:** Using `useParams()` guarantees that when any user opens a shared URL link directly or reloads the page, React extracts the exact same room ID and re-establishes the socket subscription automatically.
- 3. Resource Cleanup:** The `useEffect` cleanup function (`return () => socket.disconnect()`) prevents memory leaks, duplicate message listeners, and dangling socket connections when users leave the room.